Java & Spring Boot Interview Questions & Answers

Complete Answers with Definitions, Usage & Code Examples

Java Core • Collections • Streams • Multi-threading

Spring Boot • Spring Annotations • REST API • Hibernate / JPA

Microservices • OOP Concepts • Exception Handling • Programs

SECTION 1 – Java Core & OOP

1 What are OOP Concepts in Java?

OOP (Object-Oriented Programming) is a programming paradigm based on the concept of objects that contain data (fields) and behaviour (methods). Java has four core OOP pillars.

- **Encapsulation** – Wrapping data and methods inside a class; access via getters/setters.
- **Inheritance** – A child class inherits fields and methods from a parent class using 'extends'.
- **Polymorphism** – One name, many forms: method overloading (compile-time) and overriding (runtime).
- **Abstraction** – Hiding implementation details; show only what is necessary (abstract class / interface).

Example

```
class Animal { void sound() { System.out.println("Some sound"); } }
class Dog extends Animal {
@Override
void sound() { System.out.println("Bark"); } // Polymorphism
}
// Encapsulation
class Person {
private String name;
public String getName() { return name; }
public void setName(String n) { this.name = n; }
}
```

2 What is Polymorphism?

Polymorphism means 'many forms'. A single method or object can behave differently based on context. Java supports two types.

- **Compile-time (Static) Polymorphism** – Method Overloading: same method name, different parameters.
- **Runtime (Dynamic) Polymorphism** – Method Overriding: child class redefines parent's method.

Example

```
// Overloading (compile-time)
class Calc {
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
}
// Overriding (runtime)
class Shape { void draw() { System.out.println("Shape"); } }
class Circle extends Shape {
@Override void draw() { System.out.println("Circle"); }
}
Shape s = new Circle(); s.draw(); // prints: Circle
```

3 Difference between Method Overloading and Overriding

Both are forms of polymorphism but work very differently.

Example

```
// Overloading – same class, different params
void print(int x) {}
void print(String x) {} // different signature

// Overriding – child class changes parent behaviour
class Parent { void greet() { System.out.println("Hello"); } }
class Child extends Parent {
@Override void greet() { System.out.println("Hi!"); }
}
```

Note: You cannot override static methods; static methods are resolved at compile-time.

Aspect	Overloading	Overriding
Occurs in	Same class	Parent-child classes
Binding	Compile-time (static)	Runtime (dynamic)
Parameters	Must be different	Must be same
Return type	Can be different	Must be same or covariant
static/private	Can be overloaded	Cannot be overridden

4 Use of static keyword in Java

The 'static' keyword means a member belongs to the CLASS, not to any specific instance. Static members are shared across all objects of the class.

- **Static variable** – Shared by all instances.
- **Static method** – Can be called without creating an object.
- **Static block** – Runs once when class is loaded.
- **Static class** – Only for inner classes.

Example

```
class Counter {
static int count = 0; // shared by all instances
Counter() { count++; }
static int getCount() { return count; } // no object needed
}
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.getCount()); // 2
```

5 Types of Access Modifiers in Java

Access modifiers control the visibility of classes, methods, and fields.

Example

```
public class Demo {
public int a; // accessible everywhere
protected int b; // same package + subclasses
int c; // default: same package only
private int d; // only within this class
}
```

Note: Use private for encapsulation, public for APIs, protected for inheritance.

Modifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

6 What are final, finally and finalize?

Three very different keywords that are often confused in interviews.

- **final** – Keyword: variable = constant, method = cannot override, class = cannot inherit.
- **finally** – Block in try-catch that ALWAYS executes (cleanup code).
- **finalize()** – Method called by GC before destroying an object (deprecated in Java 9+).

Example

```
final int MAX = 100; // constant – cannot reassign
final class MyClass {} // cannot be subclassed

try {
int result = 10 / 0;
} catch (Exception e) {
System.out.println("Caught");
} finally {
System.out.println("Always runs"); // runs even on exception
}

@Override
protected void finalize() { // called by GC – avoid using
System.out.println("Object destroyed");
}
```

7 throw, throws, and Throwable – differences

All three relate to exception handling but serve different purposes.

- **throw** – Used to explicitly throw an exception inside a method.
- **throws** – Declares that a method might throw a checked exception (in signature).
- **Throwable** – The root class of all errors and exceptions in Java.

Example

```
// Throwable hierarchy:
// Throwable -> Exception -> RuntimeException (unchecked)
// -> Error (OutOfMemoryError etc.)

// throws: declares what method can throw
public void readFile(String path) throws IOException {
// throw: explicitly throws
if (path == null) throw new IllegalArgumentException("Path cannot be null");
Files.readAllLines(Path.of(path));
}
```

8 Difference between compile-time and runtime exceptions

Java exceptions are classified by when they are detected and whether they must be handled.

- **Compile-time (Checked) Exceptions** – Must be handled with try-catch or throws. Detected at compile time. Example: IOException, SQLException.
- **Runtime (Unchecked) Exceptions** – Subclass of RuntimeException. Not required to be caught. Example: NullPointerException, ArrayIndexOutOfBoundsException.
- **Errors** – Severe problems not meant to be caught. Example: OutOfMemoryError, StackOverflowError.

Example

```
// Checked – must handle at compile time
try { new FileReader("file.txt"); }
catch (FileNotFoundException e) { e.printStackTrace(); }

// Unchecked – optional handling
String s = null;
s.length(); // throws NullPointerException at runtime
```

9 Java 8 Features

Java 8 introduced major features that revolutionised Java development.

- **Lambda Expressions** – Anonymous function syntax: (a, b) -> a + b
- **Stream API** – Functional pipeline for processing collections.
- **Functional Interfaces** – Interface with exactly one abstract method (@FunctionalInterface).
- **Optional** – Wrapper to avoid NullPointerException.
- **Default/Static Interface Methods** – Methods with body inside interfaces.
- **Method References** – Shortcut for lambdas: String::toUpperCase
- **Date/Time API** – java.time.LocalDate, LocalDateTime etc.
- **Collectors** – groupingBy, toMap, joining etc.

Example

```
// Lambda + Stream + Collectors
List names = List.of("Alice", "Bob", "Charlie");
List upper = names.stream()
.filter(n -> n.length() > 3)
.map(String::toUpperCase)
.collect(Collectors.toList());
// [ALICE, CHARLIE]

// Optional
Optional opt = Optional.ofNullable(null);
String val = opt.orElse("default"); // default
```

10 What are static objects?

Static objects are class-level variables (not instance-level). A static variable or reference is created once when the class is loaded and shared across all instances.

Example

```
class Config {
// Static object – shared across all instances
static Logger logger = LoggerFactory.getLogger(Config.class);
static final Map DEFAULTS = new HashMap<>();
static {
DEFAULTS.put("timeout", "30"); // static initializer block
}
}
// Access without creating object:
Config.logger.info("Hello");
```

11 What are methods available in the Object class?

java.lang.Object is the root class of all Java classes. Every class inherits these methods.

- **equals(Object obj)** – Checks logical equality.
- **hashCode()** – Returns hash code for use in HashMap.
- **toString()** – Returns string representation.
- **getClass()** – Returns runtime class.
- **clone()** – Creates a copy (must implement Cloneable).
- **wait() / notify() / notifyAll()** – Thread synchronisation.
- **finalize()** – Called by GC before cleanup (deprecated).

Example

```
class Employee {
int id; String name;
@Override
public String toString() { return "Employee[" + id + "," + name + "]"; }
@Override
public boolean equals(Object o) {
if (!(o instanceof Employee)) return false;
return this.id == ((Employee)o).id;
}
@Override
public int hashCode() { return Integer.hashCode(id); }
}
```

Note: Object is in java.lang package – it is automatically imported.

SECTION 2 – Collections, Streams & Multi-threading

12 What are Collections? Collection interface vs Collections class

Java Collections Framework provides ready-to-use data structures.

- **Collection interface** – Root interface for List, Set, Queue. Defines add(), remove(), size(), iterator() etc.
- **Collections class** – Utility class (java.util.Collections) with static helper methods: sort(), shuffle(), min(), max(), synchronizedList() etc.

Example

```
// Collection interface – implemented by List, Set
Collection col = new ArrayList<>();
col.add("A"); col.remove("A"); col.size();

// Collections class – utility methods
List nums = Arrays.asList(3,1,4,1,5);
Collections.sort(nums); // [1,1,3,4,5]
Collections.reverse(nums); // [5,4,3,1,1]
int max = Collections.max(nums);// 5
```

13 Difference between List and Set

Both are Collection sub-interfaces but with very different behaviours.

Example

```
// List – allows duplicates, maintains order
List list = new ArrayList<>();
list.add("a"); list.add("a"); // [a, a] – duplicate kept

// Set – no duplicates, no guaranteed order
Set set = new HashSet<>();
set.add("a"); set.add("a"); // {a} – duplicate ignored

// LinkedHashSet – no duplicates BUT maintains insertion order
// TreeSet – no duplicates, sorted order
```

Feature	List	Set
Duplicates	Allowed	Not allowed
Order	Maintains insertion order	No guarantee (HashSet)
Null values	Multiple nulls allowed	Only one null (HashSet)
Index access	Yes – get(index)	No
Implementations	ArrayList, LinkedList	HashSet, LinkedHashSet, TreeSet

14 HashMap vs ConcurrentHashMap – when to choose each

Both store key-value pairs but differ in thread safety.

- **HashMap** – Not thread-safe; allows one null key; fast in single-threaded apps.
- **ConcurrentHashMap** – Thread-safe; does NOT allow null key or value; uses segment locking for high concurrency.

Example

```
// HashMap – single-threaded
Map map = new HashMap<>();
map.put("a", 1);

// ConcurrentHashMap – multi-threaded (safe to share)
Map cmap = new ConcurrentHashMap<>();
cmap.put("a", 1); // thread-safe

// WRONG: Using HashMap in multiple threads causes data corruption
// CORRECT: Use ConcurrentHashMap when multiple threads read/write
```

Note: Choose HashMap in single-thread; ConcurrentHashMap in multi-thread environments.

15 How to make List or Map synchronized (thread-safe)

You can wrap existing collections with synchronized wrappers using Collections utility class.

Example

```
// Synchronized List
List list = new ArrayList<>();
List syncList = Collections.synchronizedList(list);

// Synchronized Map
Map map = new HashMap<>();
Map syncMap = Collections.synchronizedMap(map);

// When iterating, must manually synchronize:
synchronized(syncList) {
for (String s : syncList) { System.out.println(s); }
}

// Better alternative: use ConcurrentHashMap or CopyOnWriteArrayList
List cowList = new CopyOnWriteArrayList<>();
```

16 Synchronized List vs Asynchronous Map

These are thread-safety strategies for concurrent environments.

- **Collections.synchronizedList()** – Wraps a list; all methods synchronized (blocks); iteration still needs manual sync.
- **ConcurrentHashMap** – Uses segment-based locking; multiple threads can read/write different segments simultaneously without blocking each other.
- **CopyOnWriteArrayList** – All write operations copy the array; safe iteration without synchronization; best for mostly-read lists.

Note: ConcurrentHashMap performs better than synchronizedMap because it doesn't lock the whole map.

17 Can we use synchronized block for primitive types?

No. Synchronized blocks work only with OBJECT references, not primitive types. Primitives (int, boolean, double) are not objects and cannot be used as monitor locks.

Example

```
// WRONG – cannot synchronize on primitive
// int x = 5;
// synchronized(x) {} // compile error

// CORRECT – use wrapper class (Integer is an object)
Integer lock = new Integer(1);
synchronized(lock) { /* thread-safe block */ }

// CORRECT – use AtomicInteger for thread-safe primitive operations
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet(); // thread-safe without synchronized

// CORRECT – use a dedicated lock object
private final Object lock = new Object();
synchronized(lock) { /* thread-safe */ }
```

18 Difference between Thread and Runnable

Both are ways to create threads in Java.

- **Thread class** – Extend Thread and override run(). Your class cannot extend any other class (single inheritance limitation).
- **Runnable interface** – Implement Runnable and define run(). Your class can still extend another class. Preferred approach.

Example

```
// Using Thread class (less flexible)
class MyThread extends Thread {
public void run() { System.out.println("Thread running"); }
}
new MyThread().start();

// Using Runnable (preferred)
class MyTask implements Runnable {
public void run() { System.out.println("Runnable running"); }
}
new Thread(new MyTask()).start();

// Lambda (Java 8+) – simplest
new Thread(() -> System.out.println("Lambda thread")).start();
```

19 Intermediate and Terminal Operations in Streams

Stream operations are classified by whether they produce another stream or a final result.

- **Intermediate Operations** – Return a Stream; lazy (not executed until terminal op); can be chained: filter(), map(), flatMap(), sorted(), distinct(), limit(), peek()
- **Terminal Operations** – Trigger execution and return a non-stream result: collect(), count(), forEach(), reduce(), findFirst(), anyMatch(), allMatch(), toList()

Example

```
List nums = List.of(1,2,3,4,5,6,7,8,9,10);

long result = nums.stream()
.filter(n -> n % 2 == 0) // intermediate – filter evens
.map(n -> n * n) // intermediate – square them
.sorted() // intermediate – sort
.peek(System.out::println) // intermediate – debug print
.count(); // TERMINAL – triggers execution
// Output: 4, 16, 36, 64, 100
```

20 Sequential vs Parallel Streams

Streams can process elements either one-by-one or in parallel using multiple CPU cores.

- **Sequential Stream** – Processes elements in order, single thread. Default behaviour.
- **Parallel Stream** – Splits data into chunks, processes in parallel using ForkJoinPool. Faster for large data but order not guaranteed.

Example

```
List list = List.of(1,2,3,4,5,6,7,8,9,10);

// Sequential – single thread, ordered
list.stream().forEach(System.out::print); // 12345678910

// Parallel – multiple threads, order not guaranteed
list.parallelStream().forEach(System.out::print); // 65341278910 (varies)

// Use parallel when:
// - Large data (thousands+ elements)
// - CPU-intensive operations
// - Order does not matter

// Avoid parallel when:
// - Small data (overhead > benefit)
// - Stateful operations or shared mutable state
```

21 How to find the second highest salary using Streams

A classic Java 8 streams interview question using sorted(), distinct(), and skip().

Example

```
List employees = List.of(
new Employee("Alice", 80000),
new Employee("Bob", 95000),
new Employee("Carol", 95000),
new Employee("Dave", 70000)
);

// Method 1: using sorted + distinct + skip
Optional secondHighest = employees.stream()
.map(Employee::getSalary)
.distinct() // remove duplicates: 80000,95000,70000
.sorted(Comparator.reverseOrder()) // 95000,80000,70000
.skip(1) // skip highest
.findFirst(); // 80000

secondHighest.ifPresent(s -> System.out.println("2nd highest: " + s));
// Output: 2nd highest: 80000.0
```

22 Count occurrence of characters in a given string

A common Java program to count how many times each character appears in a string.

Example

```
String input = "programming";

// Method 1: Using HashMap
Map charCount = new HashMap<>();
for (char c : input.toCharArray()) {
charCount.put(c, charCount.getOrDefault(c, 0L) + 1);
}
System.out.println(charCount); // {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}

// Method 2: Using Java 8 Streams (one-liner)
Map result = input.chars()
.mapToObj(c -> (char) c)
.collect(Collectors.groupingBy(
c -> c, Collectors.counting()));
System.out.println(result);
```

23 How to reverse a string without using predefined methods

Using a manual loop to reverse character by character.

Example

```
public static String reverse(String str) {
char[] chars = str.toCharArray();
int left = 0, right = chars.length - 1;
while (left < right) {
// Swap characters
char temp = chars[left];
chars[left] = chars[right];
chars[right] = temp;
left++;
right--;
}
return new String(chars);
}
System.out.println(reverse("Hello")); // olleH

// Recursive approach:
public static String reverseRec(String s) {
if (s.isEmpty()) return s;
return reverseRec(s.substring(1)) + s.charAt(0);
}
```

24 Check whether a string is an anagram

Two strings are anagrams if they contain the same characters in any order.

Example

```
public static boolean isAnagram(String s1, String s2) {
if (s1.length() != s2.length()) return false;
char[] a = s1.toLowerCase().toCharArray();
char[] b = s2.toLowerCase().toCharArray();
Arrays.sort(a);
Arrays.sort(b);
return Arrays.equals(a, b);
}
System.out.println(isAnagram("listen", "silent")); // true
System.out.println(isAnagram("hello", "world")); // false

// Using Streams:
boolean anagram = s1.chars().sorted().boxed().collect(Collectors.toList())
.equals(s2.chars().sorted().boxed().collect(Collectors.toList()));
```

25 What are classes in the java.util package?

The java.util package contains Java's most commonly used utility classes.

- **Collections Framework** – ArrayList, LinkedList, HashMap, HashSet, TreeMap, LinkedHashMap, PriorityQueue, ArrayDeque
- **Utility classes** – Collections, Arrays, Objects
- **Date/Time (old)** – Date, Calendar
- **Other** – Scanner, Random, UUID, Optional, StringTokenizer

Example

```
import java.util.*; // imports all util classes

List list = new ArrayList<>();
Map map = new HashMap<>();
Set set = new HashSet<>();
Queue queue = new LinkedList<>();
Optional opt = Optional.of("hello");
Scanner sc = new Scanner(System.in);
Random rnd = new Random();
```

26 Write a program to check prime number

A prime number is divisible only by 1 and itself.

Example

```
public static boolean isPrime(int n) {
if (n <= 1) return false; // 0,1 not prime
if (n <= 3) return true; // 2,3 are prime
if (n % 2 == 0 || n % 3 == 0) return false;
// Check divisors from 5 to sqrt(n)
for (int i = 5; i * i <= n; i += 6) {
if (n % i == 0 || n % (i + 2) == 0) return false;
}
return true;
}

System.out.println(isPrime(2)); // true
System.out.println(isPrime(7)); // true
System.out.println(isPrime(10)); // false
System.out.println(isPrime(97)); // true
```

SECTION 3 – Spring Boot

27 What is Spring Framework? IoC and DI

Spring is a comprehensive Java framework for building enterprise applications.

- **IoC (Inversion of Control)** – Control of object creation is transferred from your code to the Spring container.
- **DI (Dependency Injection)** – Spring injects required dependencies into a class instead of the class creating them.

Example

```
// Without DI (tight coupling – bad)
class OrderService {
PaymentService ps = new PaymentService(); // hard-coded
}

// With DI (loose coupling – good)
@Service
class OrderService {
private final PaymentService paymentService;
// Spring injects PaymentService automatically
public OrderService(PaymentService paymentService) {
this.paymentService = paymentService;
}
}
```

28 Advantages of Spring Boot

Spring Boot simplifies Spring application development with opinionated defaults.

- **Auto-configuration** – Automatically configures beans based on classpath.
- **Embedded server** – Ships with Tomcat; no external deployment needed.
- **Starter POMs** – Curated dependency sets (spring-boot-starter-web etc.).
- **Production-ready** – Actuator for health, metrics, monitoring.
- **No XML config** – Everything via annotations and application.properties.
- **Standalone JAR** – Run with java -jar; no web server required separately.
- **DevTools** – Hot reload during development.

Note: Spring Boot 3.x requires Java 17+ and uses Jakarta EE namespace.

29 Disadvantages of Spring Boot

Spring Boot is powerful but not without limitations.

- Large JAR size due to bundled dependencies.
- Auto-configuration can be hard to debug/override when it behaves unexpectedly.
- Memory footprint is heavier than lightweight frameworks like Micronaut or Quarkus.
- Startup time slower than native or reactive frameworks for microservices.
- Over-engineering risk – adding too many starters for simple applications.

30 How to change the server port number

Spring Boot starts on port 8080 by default. Change it in multiple ways.

Example

```
# Method 1: application.properties
server.port=9090

# Method 2: application.yml
server:
port: 9090

# Method 3: Command-line argument
java -jar myapp.jar --server.port=9090

# Method 4: Environment variable
SERVER_PORT=9090 java -jar myapp.jar

# Method 5: In main class
SpringApplication app = new SpringApplication(MyApp.class);
app.setDefaultProperties(Map.of("server.port","9090"));
app.run(args);
```

31 What is @SpringBootApplication?

It is a meta-annotation that combines three important annotations into one.

- **@SpringBootConfiguration** – Marks this class as a configuration source (same as @Configuration).
- **@EnableAutoConfiguration** – Triggers Spring Boot's auto-configuration mechanism.
- **@ComponentScan** – Scans the current package and sub-packages for Spring components.

Example

```
@SpringBootApplication // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class MyApp {
public static void main(String[] args) {
SpringApplication.run(MyApp.class, args);
}
}

// Equivalent to:
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(basePackages = "com.example")
public class MyApp { ... }
```

Note: @SpringBootApplication is the FIRST annotation to execute – it bootstraps the whole application.

32 How does @AutoConfiguration work and how to exclude it

Auto-configuration automatically creates beans based on the JARs on the classpath.

- Spring Boot reads META-INF/spring/AutoConfiguration.imports from every JAR.
- Each listed class has @Conditional annotations that check if beans should be created.
- @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty etc.
- User-defined beans take priority over auto-configured ones.

Example

```
// Exclude specific auto-configuration
@SpringBootApplication(exclude = {
DataSourceAutoConfiguration.class,
SecurityAutoConfiguration.class
})
public class MyApp { ... }

# Or in application.properties:
spring.autoconfigure.exclude=\
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration

# Debug auto-configuration decisions:
debug=true # shows CONDITIONS EVALUATION REPORT at startup
```

33 What is @ComponentScan?

@ComponentScan tells Spring where to look for components (classes annotated with @Component, @Service, @Repository, @Controller).

Example

```
// Default: scans current package and sub-packages
@SpringBootApplication
public class MyApp {} // scans com.example.*

// Custom scan packages
@SpringBootApplication(
scanBasePackages = {"com.example.service", "com.example.api"})
public class MyApp {}

// Exclude specific classes
@ComponentScan(
excludeFilters = @ComponentScan.Filter(
type = FilterType.ASSIGNABLE_TYPE,
classes = TestConfig.class))
```

34 What are different Bean Scopes in Spring?

Bean scope defines how many instances Spring creates and how long they live.

Example

```
@Component
@Scope("singleton") // default – one instance per context
public class OrderService {}

@Component
@Scope("prototype") // new instance every time it is requested
public class ShoppingCart {}

// Web scopes (Spring MVC only)
@Component @RequestScope // one per HTTP request
public class RequestData {}

@Component @SessionScope // one per HTTP session
public class UserSession {}
```

Scope	Instances Created	Use Case
singleton	One per Spring container (default)	Stateless services, DAOs
prototype	New instance every request	Stateful objects, shopping cart
request	One per HTTP request	Request-specific data
session	One per HTTP session	User session data
application	One per ServletContext	App-wide shared state

35 What is Global Scope / Application Scope in Spring?

Application scope means one single bean instance per ServletContext (the entire web application). It is similar to singleton but specifically for web applications and is tied to the lifecycle of the ServletContext, not the Spring ApplicationContext.

Example

```
@Component
@ApplicationScope // or @Scope("application")
public class AppConfiguration {
private Map globalSettings = new HashMap<>();
// shared across ALL users and ALL requests
}

// Note: singleton and application scope are very similar.
// singleton = one per Spring ApplicationContext
// application = one per web ServletContext
```

36 BeanFactory vs ApplicationContext

Both are Spring IoC containers but ApplicationContext is the recommended one.

- **BeanFactory** – Basic container; lazy initialisation (creates beans on demand); lightweight; no AOP, events, or i18n.
- **ApplicationContext** – Advanced container; eager initialisation; supports AOP, events, internationalisation, annotation config.

Example

```
// BeanFactory (rarely used directly)
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("beans.xml"));
MyBean bean = factory.getBean(MyBean.class);

// ApplicationContext (preferred)
ApplicationContext ctx =
new AnnotationConfigApplicationContext(AppConfig.class);
MyBean bean = ctx.getBean(MyBean.class);

// In Spring Boot, ApplicationContext is auto-created
SpringApplication.run(MyApp.class, args); // creates ApplicationContext
```

Note: ApplicationContext is a superset of BeanFactory. Always use ApplicationContext.

37 What is Spring Context?

Spring ApplicationContext is the central container that holds all configured beans, manages their lifecycle, and provides all Spring services (DI, AOP, events, i18n).

Example

```
// Access ApplicationContext directly (rare)
@Component
public class MyComponent implements ApplicationContextAware {
private ApplicationContext ctx;

@Override
public void setApplicationContext(ApplicationContext ctx) {
this.ctx = ctx;
}

public void doSomething() {
// Get bean by type
OrderService svc = ctx.getBean(OrderService.class);
// Get bean by name
Object b = ctx.getBean("orderService");
}
}
```

38 What are @Component and @Bean?

Both register beans in the Spring context but in different ways.

- **@Component** – Class-level annotation; Spring auto-detects via @ComponentScan.
- **@Bean** – Method-level annotation inside @Configuration class; you control creation.

Example

```
// @Component – Spring auto-creates this bean
@Component
public class EmailService {
public void send(String to) { ... }
}
// @Bean – manually create and configure
@Configuration
public class AppConfig {
@Bean
public DataSource dataSource() {
DriverManagerDataSource ds = new DriverManagerDataSource();
ds.setUrl("jdbc:h2:mem:testdb");
return ds;
}
}
// Use @Bean when you need to configure third-party classes
// that you cannot annotate with @Component
```

39 What are Stereotype Annotations?

Stereotype annotations are specialised @Component annotations that denote the role of a class.

- **@Component** – Generic Spring-managed component.
- **@Service** – Business logic layer; no extra behaviour but better readability.
- **@Repository** – Data access layer; enables persistence exception translation.
- **@Controller** – MVC controller; handles HTTP requests returning views.
- **@RestController** – @Controller + @ResponseBody; returns JSON/XML data.

Example

```
@Repository
public class UserRepository { /* DAO operations */ }

@Service
public class UserService { /* business logic */ }

@RestController
@RequestMapping("/api/users")
public class UserController { /* REST endpoints */ }

// Without @Service annotation, the class is NOT managed by Spring
// and cannot use @Autowired injection into it
```

40 What happens if we write @Service vs if we don't annotate?

@Service (and other stereotype annotations) register the class as a Spring bean. Without the annotation, Spring does not know about the class.

Example

```
// WITHOUT @Service – Spring does NOT manage this
public class PaymentService {
public void processPayment() { ... }
}
// Cannot @Autowired into another bean
// Cannot use @Transactional
// Cannot inject dependencies

// WITH @Service – Spring MANAGES this bean
@Service
public class PaymentService {
@Autowired // Spring injects dependencies
private PaymentRepository repo;
@Transactional // transaction management works
public void processPayment() { ... }
}
```

41 Without any annotation how can you make a class work in Spring?

You can manually register a class as a bean using @Bean in a @Configuration class.

Example

```
// Plain class – no annotations
public class ExternalEmailSender {
public void send(String to, String msg) { ... }
}

// Manually register via @Bean
@Configuration
public class AppConfig {
@Bean
public ExternalEmailSender emailSender() {
return new ExternalEmailSender();
}
}

// Now it can be injected anywhere
@Service
public class NotifyService {
@Autowired
private ExternalEmailSender emailSender; // works!
}
```

42 What is @Qualifier and @Primary?

When multiple beans of the same type exist, Spring doesn't know which to inject. @Qualifier and @Primary resolve this ambiguity.

- **@Primary** – Marks one bean as the default candidate when no qualifier is specified.
- **@Qualifier("name")** – Explicitly specifies which bean to inject by name.

Example

```
interface PaymentGateway { void charge(double amount); }

@Component @Primary // default – used when no qualifier specified
public class StripeGateway implements PaymentGateway { ... }

@Component
public class PaypalGateway implements PaymentGateway { ... }

@Service
public class OrderService {
@Autowired // injects StripeGateway (Primary)
private PaymentGateway defaultGateway;

@Autowired
@Qualifier("paypalGateway") // explicitly choose PayPal
private PaymentGateway paypalGateway;
}
```

43 What is the default server in Spring Boot? How to change it?

Spring Boot uses Tomcat as the default embedded web server.

Example

```
org.springframework.boot
spring-boot-starter-web

org.springframework.boot
spring-boot-starter-tomcat


org.springframework.boot
spring-boot-starter-jetty
```

Note: Supported servers: Tomcat (default), Jetty, Undertow. All are embedded.

44 What are the various ways to connect to a database in Spring Boot?

Spring Boot provides multiple ways to connect and interact with a database.

- **Spring Data JPA** – JpaRepository interface; auto-generates queries.
- **Spring JDBC** – JdbcTemplate; write raw SQL with less boilerplate.
- **Spring Data MongoDB/Redis/etc.** – Same Repository pattern for NoSQL.
- **R2DBC** – Reactive non-blocking database access.

Example

```
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update

// JPA Repository – simplest
public interface UserRepository extends JpaRepository {
List findByEmail(String email);
}

// JdbcTemplate – raw SQL
@Autowired JdbcTemplate jdbc;
List users = jdbc.query(
"SELECT * FROM users WHERE active=?",
(rs, row) -> new User(rs.getLong("id"), rs.getString("name")),
true);
```

45 Can we create a standalone application using Spring Boot?

Yes! That is one of Spring Boot's key features. You can run it as a plain Java JAR.

Example

```
// Standalone non-web Spring Boot application
@SpringBootApplication
public class BatchApp implements CommandLineRunner {
@Autowired
private ReportService reportService;

public static void main(String[] args) {
SpringApplication.run(BatchApp.class, args);
}

@Override
public void run(String... args) throws Exception {
// Runs after application context is ready
reportService.generateMonthlyReport();
System.out.println("Report generated!");
}
}

# Build and run:
mvn package
java -jar myapp.jar
```

SECTION 4 – REST API & Spring Annotations

46 What is @Controller and @RestController?

Both handle HTTP requests but differ in what they return.

- **@Controller** – Returns a view name (HTML page via Thymeleaf/JSP). Used for MVC apps.
- **@RestController** – Returns data directly as JSON/XML. = @Controller + @ResponseBody.

Example

```
// @Controller – returns HTML view
@Controller
public class PageController {
@GetMapping("/home")
public String home(Model model) {
model.addAttribute("msg", "Hello");
return "home"; // resolves to templates/home.html
}
}

// @RestController – returns JSON
@RestController
@RequestMapping("/api/users")
public class UserController {
@GetMapping("{id}")
public User getUser(@PathVariable Long id) {
return userService.findById(id); // auto-converted to JSON
}
}
```

47 What is the default return type in @RestController?

The default return type is JSON, serialised using Jackson library.

Example

```
@RestController
public class ProductController {
// Returns JSON by default: {"id":1,"name":"Laptop","price":999.99}
@GetMapping("/api/products/{id}")
public Product getProduct(@PathVariable Long id) {
return new Product(1L, "Laptop", 999.99);
}

// Returns String directly
@GetMapping("/api/status")
public String status() {
return "OK"; // returns plain text string
}

// Returns ResponseEntity for full control
@GetMapping("/api/products")
public ResponseEntity> getAll() {
return ResponseEntity.ok(productService.findAll());
}
}
```

Note: Add jackson-dataformat-xml to also support XML output.

48 What is @PathVariable, @RequestMapping, @RequestBody and @RequestParam?

These annotations extract different parts of an HTTP request.

Example

```
// @RequestMapping – maps URL to method (base URL)
@RestController
@RequestMapping("/api/orders") // base path
public class OrderController {

// @PathVariable – extracts value from URL path
// GET /api/orders/42
@GetMapping("/{id}")
public Order getById(@PathVariable Long id) {
return orderService.findById(id);
}

// @RequestParam – extracts query parameter
// GET /api/orders?status=PENDING&page;=0
@GetMapping
public List search(
@RequestParam String status,
@RequestParam(defaultValue="0") int page) {
return orderService.findByStatus(status, page);
}

// @RequestBody – extracts JSON body
// POST /api/orders body: {"customerId":"123","items":[...]}
@PostMapping
public Order create(@RequestBody CreateOrderRequest req) {
return orderService.create(req);
}
}
```

49 Which annotation methods are used for CRUD operations?

Spring MVC provides shortcut mapping annotations for each HTTP method.

Example

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

@GetMapping // READ all – GET /api/products
public List getAll() { ... }

@GetMapping("/{id}") // READ one – GET /api/products/1
public Product getById(@PathVariable Long id) { ... }

@PostMapping // CREATE – POST /api/products
public Product create(@RequestBody Product p) { ... }

@PutMapping("/{id}") // UPDATE all fields – PUT /api/products/1
public Product update(@PathVariable Long id, @RequestBody Product p) { ... }

@PatchMapping("/{id}") // UPDATE partial – PATCH /api/products/1
public Product patch(@PathVariable Long id, @RequestBody Map fields) { ... }

@DeleteMapping("/{id}") // DELETE – DELETE /api/products/1
public void delete(@PathVariable Long id) { ... }
}
```

50 Why return ResponseBody instead of just String?

ResponseEntity gives complete control over the HTTP response: status code, headers, and body.

Example

```
// Just returning String – no control over status or headers
@GetMapping("/status")
public String status() { return "OK"; }

// ResponseEntity – full control
@PostMapping("/users")
public ResponseEntity create(@RequestBody User user) {
User saved = userService.save(user);
URI location = URI.create("/api/users/" + saved.getId());
return ResponseEntity
.created(location) // 201 Created
.header("X-User-Id", saved.getId().toString())
.body(saved); // JSON body
}

@GetMapping("/users/{id}")
public ResponseEntity get(@PathVariable Long id) {
return userService.findById(id)
.map(ResponseEntity::ok) // 200 with body
.orElse(ResponseEntity.notFound().build()); // 404
}
```

51 How to return XML from a REST API?

Add jackson-dataformat-xml dependency and use 'produces' attribute.

Example

```
com.fasterxml.jackson.dataformat
jackson-dataformat-xml

// Endpoint that returns XML
@GetMapping(value="/api/products/{id}",
produces = MediaType.APPLICATION_XML_VALUE)
public Product getAsXml(@PathVariable Long id) {
return productService.findById(id);
}

// Or support both JSON and XML:
@GetMapping(value="/api/products/{id}",
produces = {
MediaType.APPLICATION_JSON_VALUE,
MediaType.APPLICATION_XML_VALUE
})
public Product get(@PathVariable Long id) { ... }
// Client chooses via: Accept: application/xml
```

52 What are consumes and produces in REST API?

These specify the data format a method accepts and returns.

Example

```
@RestController
public class FileController {

// consumes = what Content-Type the method ACCEPTS
@PostMapping(
value = "/api/upload",
consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public String upload(@RequestParam MultipartFile file) {
return "Uploaded: " + file.getOriginalFilename();
}

// produces = what Content-Type the method RETURNS
@GetMapping(
value = "/api/report",
produces = MediaType.APPLICATION_PDF_VALUE)
public byte[] getPdfReport() {
return reportService.generatePdf();
}

// Both together
@PostMapping(
value = "/api/data",
consumes = MediaType.APPLICATION_JSON_VALUE,
produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseDTO process(@RequestBody RequestDTO req) { ... }
}
```

53 How to handle exceptions in Spring Boot? @ExceptionHandler & @ControllerAdvice

Spring Boot provides global exception handling via @ControllerAdvice.

Example

```
// Custom exception
public class ResourceNotFoundException extends RuntimeException {
public ResourceNotFoundException(String msg) { super(msg); }
}

// Error response DTO
record ErrorResponse(int status, String message, LocalDateTime time) {}

// Global exception handler
@RestControllerAdvice
public class GlobalExceptionHandler {

@ExceptionHandler(ResourceNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public ErrorResponse handleNotFound(ResourceNotFoundException ex) {
return new ErrorResponse(404, ex.getMessage(), LocalDateTime.now());
}

@ExceptionHandler(MethodArgumentNotValidException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public ErrorResponse handleValidation(MethodArgumentNotValidException ex) {
String msg = ex.getBindingResult().getFieldErrors().stream()
.map(e -> e.getField() + ": " + e.getDefaultMessage())
.collect(Collectors.joining(", "));
return new ErrorResponse(400, msg, LocalDateTime.now());
}

@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponse handleGeneral(Exception ex) {
return new ErrorResponse(500, "Internal error", LocalDateTime.now());
}
}
```

54 How to handle Validations in Spring Boot?

Use Bean Validation annotations with @Valid on controller parameters.

Example

```
// Entity with validation annotations
public class CreateUserRequest {
@NotBlank(message = "Name is required")
@Size(min = 2, max = 50)
private String name;

@NotBlank @Email(message = "Valid email required")
private String email;

@Min(18) @Max(100)
private int age;

@NotNull @Positive
private Double salary;
}

// Controller triggers validation with @Valid
@PostMapping("/api/users")
public ResponseEntity create(@RequestBody @Valid CreateUserRequest req) {
return ResponseEntity.created(...).body(userService.create(req));
}
// If validation fails -> MethodArgumentNotValidException -> 400 Bad Request
```

55 What is the use of application.properties?

application.properties (or application.yml) is Spring Boot's central configuration file.

Example

```
# Server config
server.port=8080
server.servlet.context-path=/api

# Database
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=admin
spring.datasource.password=secret

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Actuator
management.endpoints.web.exposure.include=health,info,metrics

# Custom properties
app.jwt.secret=mySecretKey
app.max-retries=3

# Profiles
spring.profiles.active=dev
```

56 What does pom.xml contain?

pom.xml (Project Object Model) is the Maven build configuration file.

- **Project info** – groupId, artifactId, version, packaging.
- **Parent** – spring-boot-starter-parent for managed dependency versions.
- **Dependencies** – All libraries the project uses.
- **Build plugins** – Maven plugins for compile, test, package.
- **Properties** – Java version, Spring version, custom variables.
- **Profiles** – Environment-specific build configurations.

Example

```
spring-boot-starter-parent
3.2.0

com.example
my-app
1.0.0

org.springframework.boot
spring-boot-starter-web

org.springframework.boot
spring-boot-starter-data-jpa
```

57 What is layered architecture in Spring?

Spring Boot applications follow a standard layered architecture.

- **Presentation Layer** – @Controller / @RestController; handles HTTP requests/responses.
- **Service Layer** – @Service; business logic, transactions.
- **Repository/DAO Layer** – @Repository; database interaction.
- **Model/Entity Layer** – @Entity; database table mapping.

Example

```
// Model
@Entity class Product { @Id Long id; String name; double price; }

// Repository
@Repository
interface ProductRepo extends JpaRepository {}

// Service
@Service
class ProductService {
@Autowired ProductRepo repo;
public Product save(Product p) { return repo.save(p); }
}

// Controller
@RestController @RequestMapping("/api/products")
class ProductController {
@Autowired ProductService svc;
@PostMapping
public Product create(@RequestBody Product p) { return svc.save(p); }
}
```

58 What is Actuator and its endpoints?

Spring Boot Actuator provides production-ready monitoring endpoints for your application.

Example

```
org.springframework.boot
spring-boot-starter-actuator

# application.properties
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

Note: Access at http://localhost:8080/actuator

Endpoint	URL	Description
/health	/actuator/health	App health status (UP/DOWN)

Endpoint	URL	Description
/info	/actuator/info	App info (version, description)
/metrics	/actuator/metrics	JVM, HTTP, custom metrics
/env	/actuator/env	All environment properties
/beans	/actuator/beans	All Spring beans registered
/mappings	/actuator/mappings	All @RequestMapping URLs
/loggers	/actuator/loggers	Logging levels (change at runtime)
/threaddump	/actuator/threaddump	Current thread dump

59 Difference between REST and SOAP

Two different protocols/styles for web service communication.

Example

```
// REST – simple, uses HTTP methods
GET /api/users/1 // get user
POST /api/users // create user (JSON body)
DELETE /api/users/1 // delete user

// SOAP – XML-based, verbose


1
```

Aspect	REST	SOAP
Protocol	HTTP	HTTP, SMTP, TCP
Format	JSON / XML / Text	XML only
Speed	Faster, lightweight	Slower, verbose
Standards	No strict standard	WSDL contract required
Caching	Supported	Not supported
Use Case	Web/Mobile APIs	Enterprise, banking, WSDL-based

60 How to interact with a third-party API

Use RestTemplate (older) or WebClient (reactive, modern) in Spring Boot.

Example

```
// Method 1: RestTemplate (synchronous)
@Service
public class WeatherService {
private final RestTemplate restTemplate = new RestTemplate();
private final String API_URL = "https://api.weather.com/data?city=";

public WeatherDTO getWeather(String city) {
return restTemplate.getForObject(API_URL + city, WeatherDTO.class);
}

public OrderResponse createRemoteOrder(OrderRequest req) {
HttpHeaders headers = new HttpHeaders();
headers.set("Authorization", "Bearer " + apiKey);
headers.setContentType(MediaType.APPLICATION_JSON);
HttpEntity entity = new HttpEntity<>(req, headers);
return restTemplate.postForObject(API_URL, entity, OrderResponse.class);
}
}

// Method 2: WebClient (reactive / non-blocking, recommended)
WebClient client = WebClient.create("https://api.example.com");
WeatherDTO weather = client.get()
.uri("/weather?city={city}", city)
.header("Authorization", "Bearer " + apiKey)
.retrieve()
.bodyToMono(WeatherDTO.class)
.block();
```

SECTION 5 – Hibernate & JPA

61 Difference between Hibernate and JPA

JPA is a specification; Hibernate is the most popular implementation of that specification.

- **JPA (Jakarta Persistence API)** – A specification (interface) defining how Java objects map to DB tables. It provides annotations like @Entity, @Id, @OneToMany.
- **Hibernate** – A concrete ORM framework that implements the JPA specification. It adds extra features like caching, HQL, criteria API, lazy loading.

Example

```
// JPA annotations (work with any JPA provider)
@Entity
@Table(name = "employees")
public class Employee {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;
}
// JPA repository (Spring Data JPA)
public interface EmployeeRepo extends JpaRepository {}
// Hibernate-specific (HQL – Hibernate Query Language)
Query q = session.createQuery("FROM Employee WHERE name = :name");
// Not portable to other JPA providers
```

62 Advantages of Hibernate

Hibernate simplifies database interaction significantly over raw JDBC.

- **Reduces boilerplate** – No manual SQL for CRUD; auto-generated queries.
- **Database independence** – Supports MySQL, PostgreSQL, Oracle etc. via dialects.
- **Caching** – First-level and second-level cache improve performance.
- **Lazy loading** – Loads related data only when accessed.
- **HQL** – Object-oriented query language; portable across databases.
- **Auto DDL** – Can create/update database schema from entity classes.
- **Transaction management** – Seamless integration with Spring @Transactional.

63 Caching in Hibernate

Hibernate provides two levels of cache to reduce database queries.

- **First-level cache (L1)** – Enabled by default; session-scoped; within one session, same entity is loaded only once.
- **Second-level cache (L2)** – Optional; shared across sessions; use EhCache, Infinispan, Redis; reduces DB hits across multiple requests.
- **Query cache** – Caches the results of queries; works with L2 cache.

Example

```
# Enable L2 cache in application.properties
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=\
org.hibernate.cache.ehcache.EhCacheRegionFactory

// Annotate entity to be cached
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product {
@Id Long id;
String name;
}

// First call hits DB; subsequent calls within session use L1 cache
Product p1 = repo.findById(1L); // DB query
Product p2 = repo.findById(1L); // L1 cache hit – no DB query
```

64 What is Lazy Loading in Hibernate?

Lazy loading defers loading of related entities until they are actually accessed, improving performance by not loading unnecessary data.

Example

```
@Entity
public class Department {
@Id Long id;
String name;

// LAZY: employees not loaded until accessed
@OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
private List employees;
}

// Usage
Department dept = deptRepo.findById(1L).get();
// SQL: SELECT * FROM department WHERE id=1 (employees NOT loaded)

dept.getEmployees(); // NOW employees are loaded
// SQL: SELECT * FROM employee WHERE dept_id=1

// EAGER: always loads immediately (can cause N+1 problem)
@OneToMany(fetch = FetchType.EAGER) // loads with department query
private List employees;
```

Note: Use LAZY for large collections. Be careful of LazyInitializationException outside transaction.

65 What annotation is used in JPA to represent a table? What is @Id?

Key JPA mapping annotations for entity-to-table mapping.

Example

```
@Entity // marks class as JPA entity
@Table(name = "products") // maps to 'products' table
public class Product {

@Id // marks primary key field
@GeneratedValue(strategy = GenerationType.IDENTITY) // auto-increment
private Long id;

@Column(name = "product_name", nullable = false, length = 100)
private String name;

@Column(precision = 10, scale = 2)
private BigDecimal price;

@Enumerated(EnumType.STRING)
private Category category;

@ManyToOne @JoinColumn(name = "supplier_id")
private Supplier supplier;
}
```

SECTION 6 – Microservices & Architecture

66 Difference between Microservices and Monolithic Architecture

Two fundamentally different approaches to building applications.

Example

```
// Monolithic – all in one deployable unit
my-app.jar:
- UserModule
- OrderModule
- PaymentModule
- NotificationModule
-> ONE database, ONE deployment

// Microservices – independent services
user-service:8081 -> users DB
order-service:8082 -> orders DB
payment-service:8083 -> payments DB
notification-service:8084 -> (no DB, sends emails)
api-gateway:8080 -> routes to all services
```

Aspect	Monolithic	Microservices
Deployment	Single unit	Independent per service
Scalability	Scale whole app	Scale each service independently
Technology	One stack	Polyglot
Database	Shared	Per service
Failure	One bug = all down	Isolated failures
Complexity	Simple to start	Complex infrastructure

67 Advantages and Disadvantages of Microservices

Microservices offer significant benefits but also introduce complexity.

- **Advantages**:
- Independent deployability – deploy each service separately.
- Scalability – scale only the services under load.
- Technology flexibility – each service can use different stack.
- Fault isolation – one service down doesn't crash others.
- Smaller codebases – easier to understand and maintain.
- **Disadvantages**:
- Distributed system complexity – network failures, latency, partial failures.
- Data consistency – no ACID across services (eventual consistency).
- Testing – integration testing across services is harder.
- Operational overhead – monitoring, logging, tracing for many services.
- Increased infrastructure costs.

68 What is Load Balancing?

Load balancing distributes incoming network traffic across multiple server instances to ensure no single server is overwhelmed.

- **Client-Side LB** – Client decides which instance to call (Ribbon, Spring Cloud LoadBalancer).
- **Server-Side LB** – A load balancer (Nginx, AWS ALB) distributes traffic.
- **Round Robin** – Each request goes to the next server in sequence.
- **Least Connections** – Routes to server with fewest active connections.
- **IP Hash** – Routes same client IP to same server.

Example

```
# In Spring Cloud – @LoadBalanced makes RestTemplate Eureka-aware
@Configuration
public class Config {
@Bean @LoadBalanced
public RestTemplate restTemplate() { return new RestTemplate(); }
}

@Service
public class OrderService {
@Autowired RestTemplate restTemplate;

public Product getProduct(Long id) {
// "product-service" resolved via Eureka – load balanced!
return restTemplate.getForObject(
"http://product-service/api/products/" + id, Product.class);
}
}
```

69 What is Circuit Breaker?

Circuit Breaker prevents cascading failures by stopping calls to a failing service.

- **CLOSED** – Normal state; calls pass through; failures counted.
- **OPEN** – Failure threshold exceeded; calls immediately fail fast with fallback.
- **HALF-OPEN** – After wait time; test calls allowed; if success -> CLOSED; if fail -> OPEN.

Example

```
// Resilience4j Circuit Breaker
@Service
public class ProductClient {
@CircuitBreaker(name = "productSvc", fallbackMethod = "fallback")
public ProductDTO getProduct(Long id) {
return restTemplate.getForObject(
"http://product-service/products/" + id, ProductDTO.class);
}
public ProductDTO fallback(Long id, Throwable ex) {
return new ProductDTO(id, "Unavailable", 0.0);
}
}
# application.properties
resilience4j.circuitbreaker.instances.productSvc.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.productSvc.wait-duration-in-open-state=10s
```

70 What is Idempotent and Safe HTTP Request?

Two important properties of HTTP methods.

- **Safe** – The request does NOT modify server state. GET and HEAD are safe.
- **Idempotent** – Making the same request multiple times has the same effect as making it once.

Example

```
// SAFE + IDEMPOTENT
GET /api/products/1 // reading data, no change – safe and idempotent
// IDEMPOTENT but NOT safe
PUT /api/products/1 // updating same data multiple times = same result
DELETE /api/products/1 // first delete removes it; subsequent = 404 but no change
// NOT IDEMPOTENT, NOT SAFE
POST /api/orders // creates a new order each time – not idempotent
// Idempotency key (make POST idempotent):
POST /api/payments
Header: Idempotency-Key: unique-uuid-per-request
// Server stores key; second call with same key returns same result
```

71 What is JVM?

Java Virtual Machine is the runtime environment that executes Java bytecode.

- **Class Loader** – Loads .class files into memory.
- **Bytecode Verifier** – Checks code is valid and safe.
- **JIT Compiler** – Compiles frequently-executed bytecode to native machine code.
- **Garbage Collector** – Automatically reclaims unused memory.
- **Runtime Data Areas** – Heap (objects), Stack (method calls), Method Area (class metadata).

Example

```
Java Source Code (.java)
|
v javac compiler
Java Bytecode (.class)
|
v JVM (platform-specific)
Machine Code (runs on OS)

// Write once, run anywhere because:
// .class bytecode runs on ANY JVM (Windows/Linux/Mac)
// Each OS has its own JVM implementation
```

72 What is the use of @ComponentScan and what annotations are used in Controller?

Summary of key Spring annotations for controller and component scanning.

Example

```
// @ComponentScan – tells Spring where to find components
@SpringBootApplication // includes @ComponentScan automatically
public class App { } // scans com.example.* package

// Annotations used in Controller:
@RestController // marks class as REST controller
@RequestMapping("/api") // base URL for the controller

// Method-level:
@GetMapping("/users")
@PostMapping("/users")
@PutMapping("/users/{id}")
@DeleteMapping("/users/{id}")

// Parameter-level:
@PathVariable // from URL: /users/{id}
@RequestParam // from query: /users?name=Alice
@RequestBody // from HTTP body (JSON)
@RequestHeader // from HTTP header
@Valid // trigger validation
```

73 What is Data Structures?

A data structure is a way of organising and storing data for efficient access and modification. Java provides many built-in data structures.

- **Array** – Fixed size, indexed, same type.
- **ArrayList** – Dynamic array, allows duplicates, fast random access.
- **LinkedList** – Doubly-linked, fast insert/delete, slow random access.
- **Stack** – LIFO (Last In First Out).
- **Queue** – FIFO (First In First Out).
- **HashMap** – Key-value pairs, O(1) average access.
- **TreeMap** – Sorted key-value pairs.
- **HashSet** – Unique elements, O(1) average.
- **PriorityQueue** – Elements ordered by priority.

Example

```
// Stack – LIFO
Deque stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.push(3);
stack.pop(); // 3

// Queue – FIFO
Queue queue = new LinkedList<>();
queue.offer("A"); queue.offer("B");
queue.poll(); // A
```

QUICK REFERENCE – Key Comparisons

Annotation Summary

Annotation	Layer	Purpose
@Entity	Model	Maps class to database table
@Id	Model	Primary key field
@Repository	DAO	Data access layer; exception translation
@Service	Service	Business logic layer
@RestController	Controller	@Controller + @ResponseBody (returns JSON)
@SpringBootApplication	Main	@Configuration + @EnableAutoConfig + @ComponentScan
@Autowired	Any	Inject dependency by type
@Qualifier	Any	Inject specific bean by name
@Primary	Any	Default bean when multiple candidates exist
@Component	Any	Generic Spring-managed bean
@Bean	Config	Manually create bean in @Configuration
@Transactional	Service/Repo	Wrap method in DB transaction
@Value("${...}")	Any	Inject value from application.properties
@ExceptionHandler	Controller	Handle specific exception in a class
@ControllerAdvice	Global	Global exception handler across controllers
@Valid	Controller	Trigger Bean Validation on @RequestBody